

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 5**



CONNECT TO THE INTERNET

Oleh:

Nur Hikmah

NIM. 2310817120010

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI 2025**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 5

Laporan Praktikum Pemrograman Mobile Modul 5: Connect to the Internet ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Nur Hikmah
NIM : 2310817120010

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Zulfa Auliya Akbar
NIM. 2210817210026

Muti`a Maulida S.Kom M.T.I
NIP. 19881027 201903 20 13

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1	7
A. Source Code.....	8
B. Output Program	33
C. Pembahasan	38
D. Tautan Git	56

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 1 UI List	33
Gambar 2. Screenshot Hasil Jawaban Soal 1 UI Detail Saat Menekan Tombol Detail	34
Gambar 3. Screenshot Hasil Jawaban Soal 1 Berpindah ke Aplikasi Shopee Saat Menekan Tombol Shopee.....	35
Gambar 4. Screenshot Hasil Jawaban Soal 1 UI List Tambah Favorit	36
Gambar 5. Screenshot Hasil Jawaban Soal 1 UI Halaman Favorit	37

DAFTAR TABEL

Tabel 1. Source Code Jawaban Soal 1	8
Tabel 2. Source Code Jawaban Soal 1	8
Tabel 3. Source Code Jawaban Soal 1	9
Tabel 4. Source Code Jawaban Soal 1	9
Tabel 5. Source Code Jawaban Soal 1	10
Tabel 6. Source Code Jawaban Soal 1	10
Tabel 7. Source Code Jawaban Soal 1	11
Tabel 8. Source Code Jawaban Soal 1	12
Tabel 9. Source Code Jawaban Soal 1	12
Tabel 10. Source Code Jawaban Soal 1	13
Tabel 11. Source Code Jawaban Soal 1	14
Tabel 12. Source Code Jawaban Soal 1	15
Tabel 13. Source Code Jawaban Soal 1	17
Tabel 14. Source Code Jawaban Soal 1	18
Tabel 15. Source Code Jawaban Soal 1	19
Tabel 16. Source Code Jawaban Soal 1	20
Tabel 17. Source Code Jawaban Soal 1	21
Tabel 18. Source Code Jawaban Soal 1	21
Tabel 19. Source Code Jawaban Soal 1	22
Tabel 20. Source Code Jawaban Soal 1	22
Tabel 21. Source Code Jawaban Soal 1	23
Tabel 22. Source Code Jawaban Soal 1	23
Tabel 23. Source Code Jawaban Soal 1	24
Tabel 24. Source Code Jawaban Soal 1	26
Tabel 25. Source Code Jawaban Soal 1	27
Tabel 26. Source Code Jawaban Soal 1	28
Tabel 27. Source Code Jawaban Soal 1	28
Tabel 28. Source Code Jawaban Soal 1	29
Tabel 29. Source Code Jawaban Soal 1	29

Tabel 30. Source Code Jawaban Soal 1	32
--	----

SOAL 1

Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a. Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
- b. Gunakan KotlinX Serialization sebagai library JSON.
- c. Gunakan library seperti Coil atau Glide untuk image loading.
- d. API yang digunakan pada modul ini bebas, contoh API gratis The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API: <https://developer.themoviedb.org/docs/getting-started>
- e. Implementasikan konsep data persistence (misalnya offline-first app, pengaturan dark/light mode, fitur favorite, dll)
- f. Gunakan caching strategy pada Room.
- g. Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya.

A. Source Code

1. app/src/main/java/com/example/modul5/data/api/ApiClient.kt

```
1 package com.example.modul5.data.api
2
3 import
4   com.jakewharton.retrofit2.converter.kotlinx.serialization.
5   asConverterFactory
6 import kotlinx.serialization.json.Json
7 import okhttp3.MediaType.Companion.toMediaType
8 import retrofit2.Retrofit
9
10 object ApiClient {
11     private const val BASE_URL =
12     "https://modul5.free.beeceptor.com"
13
14     private val json = Json { ignoreUnknownKeys = true }
15
16     val retrofit: TasApiService = Retrofit.Builder()
17       .baseUrl(BASE_URL)
18       .addConverterFactory(json.asConverterFactory
19       ("application/json".toMediaType()))
20       .build()
21       .create(TasApiService::class.java)
22 }
```

Tabel 1. Source Code Jawaban Soal 1

2. app/src/main/java/com/example/modul5/data/api/TasApiService.kt

```
1 package com.example.modul5.data.api
2
3 import com.example.modul5.domain.model.Tas
4 import retrofit2.http.GET
5
6 interface TasApiService {
7     @GET("data")
8     suspend fun getTas(): List<TasDto>
9 }
```

Tabel 2. Source Code Jawaban Soal 1

3. app/src/main/java/com/example/modul5/data/api/TasDto.kt

```
1 package com.example.modul5.data.api
2
3 import kotlinx.serialization.Serializable
4
5 @Serializable
6 data class TasDto(
7     val id: Int,
8     val nama: String,
```

9	val harga: String,
10	val gambar: String,
11	val shopeeLink: String,
12	val deskripsi: String
13	)

Tabel 3. Source Code Jawaban Soal 1

4. app/src/main/java/com/example/modul5/data/local/TasDao.kt

1	package com.example.modul5.data.local
2	
3	import androidx.room.Dao
4	import androidx.room.Insert
5	import androidx.room.OnConflictStrategy
6	import androidx.room.Query
7	import com.example.modul5.data.local.TasEntity
8	import kotlinx.coroutines.flow.Flow
9	
10	@Dao
11	interface TasDao {
12	@Query("SELECT * FROM tas")
13	fun getAllTas(): Flow<List<TasEntity>>
14	
15	@Insert(onConflict = OnConflictStrategy.REPLACE)
16	suspend fun insertAll(tas: List<TasEntity>)
17	
18	@Query("UPDATE tas SET isFavorite = :isFav WHERE id =
19	:id")
20	suspend fun updateFavorite(id: Int, isFav: Boolean)
20	}

Tabel 4. Source Code Jawaban Soal 1

5. app/src/main/java/com/example/modul5/data/local/TasDatabase.kt

1	package com.example.modul5.data.local
2	
3	import android.content.Context
4	import androidx.room.Database
5	import androidx.room.Room
6	import androidx.room.RoomDatabase
7	
8	@Database(entities = [TasEntity::class], version = 1,
9	exportSchema = false)
9	abstract class TasDatabase : RoomDatabase() {
10	abstract fun tasDao(): TasDao
11	
12	companion object {
13	@Volatile
14	private var INSTANCE: TasDatabase? = null
15	

16	fun getDatabase(context: Context): TasDatabase {
17	return INSTANCE ?: synchronized(this) {
18	val instance = Room.databaseBuilder(
19	context.applicationContext,
20	TasDatabase::class.java,
21	"tas_db"
22	).build()
23	INSTANCE = instance
24	instance
25	}
26	}
27	}
28	}

Tabel 5. Source Code Jawaban Soal 1

6. app/src/main/java/com/example/modul5/data/local/TasEntity.kt

1	package com.example.modul5.data.local
2	
3	import androidx.room.Entity
4	import androidx.room.PrimaryKey
5	
6	@Entity(tableName = "tas")
7	data class TasEntity(
8	@PrimaryKey val id: Int,
9	val nama: String,
10	val harga: String,
11	val gambar: String,
12	val shopeeLink: String,
13	val deskripsi: String,
14	val isFavorite: Boolean = false
15	)

Tabel 6. Source Code Jawaban Soal 1

7. app/src/main/java/com/example/modul5/data/mapper/TasMapper.kt

1	package com.example.modul5.data.mapper
2	
3	import com.example.modul5.data.api.TasDto
4	import com.example.modul5.data.local.TasEntity
5	import com.example.modul5.domain.model.Tas
6	
7	fun TasDto.toEntity(): TasEntity {
8	return TasEntity(
9	id = this.id,
10	nama = this.nama,
11	harga = this.harga,
12	gambar = this.gambar,
13	shopeeLink = this.shopeeLink,
14	deskripsi = this.deskripsi,

15	isFavorite = false // default
16	)
17	}
18	
19	fun TasEntity.toDomain(): Tas {
20	return Tas(
21	id = this.id,
22	nama = this.nama,
23	harga = this.harga,
24	gambar = this.gambar,
25	shopeeLink = this.shopeeLink,
26	deskripsi = this.deskripsi,
27	isFavorite = this.isFavorite
28	)
29	}

Tabel 7. Source Code Jawaban Soal 1

8. app/src/main/java/com/example/modul5/data/repository/TasRepository.kt

1	package com.example.modul5.data.repository
2	
3	import android.util.Log
4	import com.example.modul5.domain.model.Tas
5	import com.example.modul5.data.api.TasApiService
6	import com.example.modul5.data.local.TasDao
7	import com.example.modul5.data.mapper.toDomain
8	import com.example.modul5.data.mapper.toEntity
9	import kotlinx.coroutines.flow.Flow
10	import kotlinx.coroutines.flow.firstOrNull
11	import kotlinx.coroutines.flow.map
12	
13	class TasRepository(
14	private val api: TasApiService,
15	private val dao: TasDao
16	) {
17	val tasList: Flow<List<Tas>> = dao.getAllTas().map { list
18	->
19	list.map { it.toDomain() }
20	}
21	suspend fun refreshFromApi() {
22	val response = api.getTas()
23	val existing = dao.getAllTas().firstOrNull()
24	if (existing.isNullOrEmpty()) {
25	dao.insertAll(response.map { it.toEntity() })
26	}
27	}
28	
29	suspend fun toggleFavorite(id: Int, current: Boolean) {
30	dao.updateFavorite(id, !current)
31	}

32	}
----	---

Tabel 8. Source Code Jawaban Soal 1

9. app/src/main/java/com/example/modul5/domain/model/Tas.kt

1	package com.example.modul5.domain.model
2	
3	import android.os.Parcelable
4	import kotlinx.parcelize.Parcelize
5	import kotlinx.serialization.Serializable
6	import kotlinx.serialization.Transient
7	
8	@Serializable
9	@Parcelize
10	data class Tas(
11	val id: Int,
12	val nama: String,
13	val harga: String,
14	val gambar: String,
15	val shopeeLink: String,
16	val deskripsi: String,
17	@Transient val isFavorite: Boolean = false
18	) : Parcelable

Tabel 9. Source Code Jawaban Soal 1

10. app/src/main/java/com/example/modul5/ui/adapter/AdapterTas.kt

1	package com.example.modul5.ui.adapter
2	
3	import android.view.LayoutInflater
4	import android.view.ViewGroup
5	import androidx.recyclerview.widget.RecyclerView
6	import coil.load
7	import com.example.modul5.R
8	import com.example.modul5.databinding.ItemTasBinding
9	import com.example.modul5.domain.model.Tas
10	
11	class AdapterTas(
12	private val onDetailClick: (Tas) -> Unit,
13	private val onShopeeClick: (Tas) -> Unit,
14	private val onFavoriteClick: (Tas) -> Unit
15	) : RecyclerView.Adapter<AdapterTas.TasViewHolder>() {
16	private var tasList: List<Tas> = emptyList()
17	
18	fun updateList(newList: List<Tas>) {
19	tasList = newList
20	notifyDataSetChanged()
21	}
22	inner class TasViewHolder(val binding: ItemTasBinding) : RecyclerView.ViewHolder(binding.root)

23	
24	override fun onCreateViewHolder(parent: ViewGroup,
25	viewType: Int): TasViewHolder {
26	val binding =
27	ItemTasBinding.inflate(LayoutInflater.from(parent.context),
28	parent, false)
29	return TasViewHolder(binding)
30	}
31	override fun onBindViewHolder(holder: TasViewHolder,
32	position: Int) {
33	val tas = tasList[position]
34	with(holder.binding) {
35	tvItemName.text = tas.nama
36	tvItemPrice.text = tas.harga
37	imgItemPhoto.load(tas.gambar)
38	btnFavorite.setImageResource(
39	if (tas.isFavorite)
40	R.drawable.ic_love_filled else R.drawable.ic_love_border
41	)
42	btnFavorite.setOnClickListener {
43	onFavoriteClick(tas) }
44	buttonShopee.setOnClickListener {
45	onShopeeClick(tas) }
	buttonDetail.setOnClickListener {
	onDetailClick(tas) }
	}
	}
	override fun getItemCount(): Int = tasList.size
	}

Tabel 10. Source Code Jawaban Soal 1

11. app/src/main/java/com/example/modul5/ui/adapters/HomePagerAdapter.kt

1	package com.example.modul5.ui.adapters
2	
3	import androidx.fragment.app.Fragment
4	import androidx.viewpager2.adapter.FragmentStateAdapter
5	import com.example.modul5.ui.home.tabs.AllProductFragment
6	import com.example.modul5.ui.home.tabs.FavoriteFragment
7	
8	class HomePagerAdapter(fragment: Fragment) :
9	FragmentStateAdapter(fragment) {
10	override fun getItemCount(): Int = 2
11	
12	override fun createFragment(position: Int): Fragment {
13	return when (position) {
14	0 -> AllProductFragment()
15	1 -> FavoriteFragment()

16	else -> throw IllegalArgumentException("Invalid
17	position: \$position")
18	}
19	}

Tabel 11. Source Code Jawaban Soal 1

12. app/src/main/java/com/example/modul5/ui/detail/DetailFragment.kt

1	package com.example.modul5.ui.detail
2	
3	import android.os.Bundle
4	import android.util.Log
5	import android.view.LayoutInflater
6	import android.view.View
7	import android.view.ViewGroup
8	import android.widget.Toast
9	import androidx.appcompat.app.AppCompatActivity
10	import androidx.fragment.app.Fragment
11	import androidx.fragment.app.activityViewModels
12	import androidx.lifecycle.ViewModelProvider
13	import androidx.lifecycle.LifecycleScope
14	import coil.load
15	import com.example.modul5.R
16	import com.example.modul5.databinding.FragmentDetailBinding
17	import com.example.modul5.viewmodel.TasViewModel
18	import kotlinx.coroutines.flow.collectLatest
19	import kotlinx.coroutines.launch
20	import androidx.navigation.fragment.findNavController
21	import com.example.modul5.viewmodel.TasViewModelFactory
22	
23	class DetailFragment : Fragment() {
24	private val viewModel: TasViewModel by activityViewModels
25	{
26	TasViewModelFactory(requireContext())
27	}
28	private var _binding: FragmentDetailBinding? = null
29	private val binding get() = _binding!!
30	override fun onCreateView(
31	inflater: LayoutInflater, container: ViewGroup?,
32	savedInstanceState: Bundle?
33	): View {
34	_binding = FragmentDetailBinding.inflate(inflater,
35	container, false)
36	return binding.root
37	}
38	override fun onViewCreated(view: View,
39	savedInstanceState: Bundle?) {
	super.onViewCreated(view, savedInstanceState)

40	val activity = requireActivity() as AppCompatActivity
41	activity.supportActionBar(binding.toolbar)
42	activity.supportActionBar?.apply {
43	setDisplayHomeAsUpEnabled(true)
44	title = "ShopBag - Your Bag Your Style"
45	}
46	
47	binding.toolbar.setNavigationOnClickListener {
48	findNavController().navigateUp()
49	}
50	
51	lifecycleScope.launch {
52	viewModel.selectedTas.collectLatest { tas ->
53	if (tas != null) {
54	binding.tvName.text = tas.nama
55	binding.tvHarga.text =
56	getString(R.string.label_harga) + ": " + tas.harga
57	binding.tvDeskripsi.text = tas.deskripsi
58	binding.imgItemPhoto.load(tas.gambar) {
59	placeholder(R.drawable.placeholder)
60	error(R.drawable.ic_error)
61	}
62	} else {
63	findNavController().navigateUp()
64	}
65	}
66	}
67	
68	override fun onDestroyView() {
69	super.onDestroyView()
70	viewModel.clearSelectedTas()
71	_binding = null
72	}
73	}

Tabel 12. Source Code Jawaban Soal 1

13. app/src/main/java/com/example/modul5/ui/home/HomeFragment.kt

1	package com.example.modul5.ui.home
2	
3	import android.os.Bundle
4	import android.util.Log
5	import android.view.*
6	import androidx.fragment.app.Fragment
7	import androidx.fragment.app.activityViewModels
8	import androidx.lifecycle.LifecycleScope
9	import com.example.modul5.R
10	import androidx.navigation.fragment.findNavController
11	import com.example.modul5.databinding.FragmentHomeBinding
12	import com.example.modul5.ui.adapter.HomePagerAdapter

```

13 import com.example.modul5.viewmodel.TasViewModel
14 import com.example.modul5.viewmodel.TasViewModelFactory
15 import com.google.android.material.tabs.TabLayoutMediator
16 import kotlinx.coroutines.launch
17
18 class HomeFragment : Fragment() {
19     private val viewModel: TasViewModel by activityViewModels {
20         TasViewModelFactory(requireContext())
21     }
22
23     private var _binding: FragmentHomeBinding? = null
24     private val binding get() = _binding!!
25
26     private lateinit var adapter: HomePagerAdapter
27
28     override fun onCreateView(
29         inflater: LayoutInflater, container: ViewGroup?,
30         savedInstanceState: Bundle?
31     ): View {
32         _binding = FragmentHomeBinding.inflate(inflater, container,
33 false)
34         return binding.root
35     }
36
37     override fun onViewCreated(view: View, savedInstanceState: Bundle?)
38 {
39         super.onViewCreated(view, savedInstanceState)
40
41         adapter = HomePagerAdapter(this)
42         binding.viewPager.adapter = adapter
43
44         TabLayoutMediator(binding.tabLayout, binding.viewPager) { tab,
45 position ->
46         tab.text = if (position == 0) "All Product" else "Favorite"
47         }.attach()
48
49         viewLifecycleOwner.lifecycleScope.launch {
50             viewModel.navigateToDetail.collect {
51                 val selected = viewModel.selectedTas.value
52                 if (selected != null) {
53                     findNavController().navigate(R.id.action_homeFragment_to_detailFragment)
54                 } else {
55                     Log.e("HomeFragment", "navigateToDetail: selectedTas
56 is null")
57                 }
58             }
59         }
60
61         override fun onDestroyView() {
62             super.onDestroyView()
63         }
64     }
65 }

```

60	<code>_binding = null</code>
61	<code>}</code>
62	<code>}</code>

Tabel 13. Source Code Jawaban Soal 1

14. app/src/main/java/com/example/modul5/ui/home/tabs/AllProductFragment.kt

1	<code>package com.example.modul5.ui.home.tabs</code>
2	
3	<code>import android.content.Intent</code>
4	<code>import android.net.Uri</code>
5	<code>import android.os.Bundle</code>
6	<code>import android.view.*</code>
7	<code>import androidx.fragment.app.Fragment</code>
8	<code>import androidx.fragment.app.activityViewModels</code>
9	<code>import androidx.lifecycle.lifecycleScope</code>
10	<code>import androidx.recyclerview.widget.LinearLayoutManager</code>
11	<code>import</code>
12	<code>com.example.modul5.databinding.FragmentAllProductBinding</code>
13	<code>import com.example.modul5.ui.adapter.AdapterTas</code>
14	<code>import com.example.modul5.viewmodel.TasViewModel</code>
15	<code>import com.example.modul5.viewmodel.TasViewModelFactory</code>
16	<code>import kotlinx.coroutines.launch</code>
17	<code>class AllProductFragment : Fragment() {</code>
18	<code> private var _binding: FragmentAllProductBinding? = null</code>
19	<code> private val binding get() = _binding!!</code>
20	<code> private val viewModel: TasViewModel by activityViewModels</code>
21	<code>{</code>
22	<code> TasViewModelFactory(requireContext()) }</code>
23	<code> private lateinit var adapter: AdapterTas</code>
24	<code> override fun onCreateView(inflater: LayoutInflater,</code>
25	<code> container: ViewGroup?, savedInstanceState: Bundle?): View {</code>
26	<code> _binding =</code>
27	<code> FragmentAllProductBinding.inflate(inflater, container,</code>
28	<code> false)</code>
29	<code> return binding.root</code>
30	<code> }</code>
31	<code> override fun onViewCreated(view: View,</code>
32	<code> savedInstanceState: Bundle?) {</code>
33	<code> adapter = AdapterTas(</code>
34	<code> onDetailClick = { viewModel.selectTas(it) },</code>
35	<code> onShopeeClick = {</code>
36	<code> val intent = Intent(Intent.ACTION_VIEW,</code>
	<code> Uri.parse(it.shopeeLink))</code>
	<code> startActivity(intent)</code>
	<code> },</code>
	<code> onFavoriteClick = { viewModel.toggleFavorite(it)</code>
	<code> }</code>

37	)	
38	binding.recyclerViewAll.apply {	
39	layoutManager	=
	LinearLayoutManager(requireContext())	
40	adapter = this@AllProductFragment.adapter	
41	}	
42	viewLifecycleOwner.lifecycleScope.launch {	
43	viewModel.tasList.collect { list ->	
44	adapter.updateList(list)	
45	}	
46	}	
47	}	
48		
49	override fun onDestroyView() {	
50	super.onDestroyView()	
51	_binding = null	
52	}	
53	}	

Tabel 14. Source Code Jawaban Soal 1

15. app/src/main/java/com/example/modul5/ui/home/tabs/FavoriteFragment.kt

1	package com.example.modul5.ui.home.tabs
2	
3	import android.content.Intent
4	import android.net.Uri
5	import android.os.Bundle
6	import android.view.*
7	import androidx.fragment.app.Fragment
8	import androidx.fragment.app.activityViewModels
9	import androidx.lifecycle.lifecycleScope
10	import androidx.recyclerview.widget.LinearLayoutManager
11	import
	com.example.modul5.databinding.FragmentFavoriteBinding
12	import com.example.modul5.ui.adapter.AdapterTas
13	import com.example.modul5.viewmodel.TasViewModel
14	import com.example.modul5.viewmodel.TasViewModelFactory
15	import kotlinx.coroutines.launch
16	
17	class FavoriteFragment : Fragment() {
18	private var _binding: FragmentFavoriteBinding? = null
19	private val binding get() = _binding!!
20	private val viewModel: TasViewModel by activityViewModels
	{
21	TasViewModelFactory(requireContext()) }
22	private lateinit var adapter: AdapterTas
23	
24	override fun onCreateView(inflater: LayoutInflater,
	container: ViewGroup?, savedInstanceState: Bundle?): View {
25	_binding = FragmentFavoriteBinding.inflate(inflater,
	container, false)

```

26         return binding.root
27     }
28
29     override fun onViewCreated(view: View,
savedInstanceState: Bundle?) {
30         adapter = AdapterTas(
31             onDetailClick = { viewModel.selectTas(it) },
32             onShopeeClick = {
33                 val intent = Intent(Intent.ACTION_VIEW,
Uri.parse(it.shopeeLink))
34                 startActivity(intent)
35             },
36             onFavoriteClick = { viewModel.toggleFavorite(it)
}
37         )
38         binding.recyclerViewFavorite.apply {
39             layoutManager =
LinearLayoutManager(requireContext())
40             adapter = this@FavoriteFragment.adapter
41         }
42         viewLifecycleOwner.lifecycleScope.launch {
43             viewModel.tasList.collect { list ->
44                 val favorites = list.filter { it.isFavorite }
45                 adapter.updateList(favorites)
46             }
47         }
48     }
49
50     override fun onDestroyView() {
51         super.onDestroyView()
52         _binding = null
53     }
54 }

```

Tabel 15. Source Code Jawaban Soal 1

16. app/src/main/java/com/example/modul5/viewmodel/TasViewModel.kt

```

1 package com.example.modul5.viewmodel
2
3 import androidx.lifecycle.ViewModel
4 import androidx.lifecycle.viewModelScope
5 import com.example.modul5.domain.model.Tas
6 import com.example.modul5.data.repository.TasRepository
7 import kotlinx.coroutines.flow.*
8 import kotlinx.coroutines.launch
9
10 class TasViewModel(
11     private val repository: TasRepository
12 ) : ViewModel() {
13     val tasList: StateFlow<List<Tas>> =
14         repository.tasList

```

15	<code>.onEach { println("ViewModel: Jumlah data dari Room = \${it.size}") }</code>
16	<code>.stateIn(viewModelScope, SharingStarted.Lazily, emptyList())</code>
17	<code>private val _selectedTas = MutableStateFlow<Tas?>(null)</code>
18	<code>val selectedTas: StateFlow<Tas?> = _selectedTas</code>
19	<code>private val _navigateToDetail = MutableSharedFlow<Unit>()</code>
20	<code>val navigateToDetail: SharedFlow<Unit> = _navigateToDetail</code>
21	
22	<code>init {</code>
23	<code>refreshFromApi()</code>
24	<code>}</code>
25	
26	<code>fun refreshFromApi() {</code>
27	<code>viewModelScope.launch {</code>
28	<code>try {</code>
29	<code>repository.refreshFromApi()</code>
30	<code>} catch (e: Exception) {</code>
31	<code>}</code>
32	<code>}</code>
33	<code>}</code>
34	
35	<code>fun toggleFavorite(tas: Tas) {</code>
36	<code>viewModelScope.launch {</code>
37	<code>repository.toggleFavorite(tas.id, tas.isFavorite)</code>
38	<code>}</code>
39	<code>}</code>
40	
41	<code>fun selectTas(item: Tas?) {</code>
42	<code>_selectedTas.value = item</code>
43	<code>if (item != null) {</code>
44	<code>viewModelScope.launch {</code>
45	<code>_navigateToDetail.emit(Unit)</code>
46	<code>}</code>
47	<code>}</code>
48	<code>}</code>
49	
50	<code>fun clearSelectedTas() {</code>
51	<code>_selectedTas.value = null</code>
52	<code>}</code>
53	<code>}</code>

Tabel 16. Source Code Jawaban Soal 1

17. app/src/main/java/com/example/modul5/viewmodel/TasViewModelFactory.kt

1	<code>package com.example.modul5.viewmodel</code>
2	
3	<code>import android.content.Context</code>
4	<code>import androidx.lifecycle.ViewModel</code>

```

5 import androidx.lifecycle.ViewModelProvider
6 import com.example.modul5.data.api.ApiClient
7 import com.example.modul5.data.local.TasDatabase
8 import com.example.modul5.data.repository.TasRepository
9
10 class TasViewModelFactory(private val context: Context) :
11     ViewModelProvider.Factory {
12     override fun <T : ViewModel> create(modelClass:
13         Class<T>): T {
14         if
15         (modelClass.isAssignableFrom(TasViewModel::class.java)) {
16             val db = TasDatabase.getDatabase(context)
17             val dao = db.tasDao()
18             val api = ApiClient.retrofit
19             val repository = TasRepository(api, dao)
20             return TasViewModel(repository) as T
21         }
22         throw IllegalArgumentException("Unknown ViewModel
23         class: ${modelClass.name}")
24     }
25 }

```

Tabel 17. Source Code Jawaban Soal 1

18. app/src/main/java/com/example/modul5/MainActivity.kt

```

1 package com.example.modul5
2
3 import android.os.Bundle
4 import androidx.appcompat.app.AppCompatActivity
5 import com.example.modul5.databinding.ActivityMainBinding
6
7 class MainActivity : AppCompatActivity() {
8     private lateinit var binding: ActivityMainBinding
9     override fun onCreate(savedInstanceState: Bundle?) {
10         super.onCreate(savedInstanceState)
11         binding = ActivityMainBinding.inflate(layoutInflater)
12         setContentView(binding.root)
13     }
14 }

```

Tabel 18. Source Code Jawaban Soal 1

19. app/src/main/res/drawable/ic_love_border.xml

```

1 <vector
2     xmlns:android="http://schemas.android.com/apk/res/android"
3     android:width="24dp" android:height="24dp"
4     android:viewportWidth="24" android:viewportHeight="24">
5     <path
6         android:pathData="M12.1,8.64L12,8.77l-0.1,-
7         0.13C10.14,6.36 6.6,6.49 4.6,8.5c-1.96,1.96 -1.96,5.14

```

	0,7.1L12,2317.4,-7.4c1.96,-1.96 1.96,-5.14 0,-7.1c-2,-2.01 - 5.54,-2.14 -7.4,0.14z"
6	android:strokeColor="#000000"
7	android:strokeWidth="1.5"
8	android:fillColor="@android:color/transparent" />
9	</vector>

Tabel 19. Source Code Jawaban Soal 1

20. app/src/main/res/drawable/ic_love_filled.xml

1	<vector
	xmlns:android="http://schemas.android.com/apk/res/android"
2	android:width="24dp" android:height="24dp"
	android:viewportWidth="24"
3	android:viewportHeight="24">
4	<path
5	android:fillColor="#FF0000"
6	android:pathData="M12 21.35l-1.45-1.32C5.4 15.36 2
	12.28 2 8.5 2 5.42 4.42 3
7	7.5 3c1.74 0 3.41 0.81 4.5 2.09C13.09 3.81 14.76 3
8	16.5 3 19.58 3 22 5.42 22 8.5c0 3.78-3.4 6.86-8.55
9	11.54L12 21.35z"/>
10	</vector>

Tabel 20. Source Code Jawaban Soal 1

21. app/src/main/res/layout/fragment_home.xml

```
1 <LinearLayout
2   xmlns:android="http://schemas.android.com/apk/res/android"
3   android:layout_width="match_parent"
4   android:layout_height="match_parent"
5   android:orientation="vertical">
6
7   <TextView
8       android:layout_width="match_parent"
9       android:layout_height="wrap_content"
10      android:background="#D1B3FF"
11      android:padding="15dp"
12      android:text="ShopBag - Your Bag Your Style"
13      android:textStyle="bold"
14      android:textColor="@color/black"
15      android:textSize="25sp" />
16
17   <com.google.android.material.tabs.TabLayout
18       android:id="@+id/tabLayout"
19       android:layout_width="match_parent"
20       android:layout_height="wrap_content" />
21
22   <androidx.viewpager2.widget.ViewPager2
23       android:id="@+id/viewPager"
24       android:layout_width="match_parent"
25       android:layout_height="0dp"
26       android:layout_weight="1" />
27 </LinearLayout>
```

Tabel 21. Source Code Jawaban Soal 1

22. app/src/main/res/layout/fragment_all_product.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <LinearLayout
3   xmlns:android="http://schemas.android.com/apk/res/android"
4   android:layout_width="match_parent"
5   android:layout_height="match_parent"
6   android:orientation="vertical"
7   android:background="#F6F6F6">
8
9   <androidx.recyclerview.widget.RecyclerView
10       android:id="@+id/recyclerViewAll"
11       android:layout_width="match_parent"
12       android:layout_height="match_parent"
13       android:padding="4dp"
14       android:clipToPadding="false" />
15 </LinearLayout>
```

Tabel 22. Source Code Jawaban Soal 1

23. app/src/main/res/layout/fragment_favorite.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <LinearLayout
3     xmlns:android="http://schemas.android.com/apk/res/android"
4         android:layout_width="match_parent"
5         android:layout_height="match_parent"
6         android:orientation="vertical"
7         android:background="#F6F6F6">
8     <androidx.recyclerview.widget.RecyclerView
9         android:id="@+id/recyclerViewFavorite"
10        android:layout_width="match_parent"
11        android:layout_height="match_parent"
12        android:padding="4dp"
13        android:clipToPadding="false" />
14
15 </LinearLayout>
```

Tabel 23. Source Code Jawaban Soal 1

24. app/src/main/res/layout/item_tas.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <androidx.cardview.widget.CardView
3     xmlns:android="http://schemas.android.com/apk/res/android"
4     xmlns:app="http://schemas.android.com/apk/res-auto"
5     xmlns:tools="http://schemas.android.com/tools"
6     android:id="@+id/card_view"
7     android:layout_width="match_parent"
8     android:layout_height="wrap_content"
9     android:layout_gravity="center"
10    android:layout_marginStart="8dp"
11    android:layout_marginTop="4dp"
12    android:layout_marginEnd="8dp"
13    android:layout_marginBottom="4dp"
14    app:cardCornerRadius="12dp"
15    app:cardElevation="10dp">
16    <androidx.constraintlayout.widget.ConstraintLayout
17        android:layout_width="match_parent"
18        android:layout_height="wrap_content"
19        android:background="#F6F6F6"
20        android:padding="20dp">
21
22        <ImageView
23            android:id="@+id/img_item_photo"
24            android:layout_width="135dp"
25            android:layout_height="140dp"
26            android:scaleType="centerCrop"
27            app:layout_constraintStart_toStartOf="parent"
```

```

28         app:layout_constraintTop_toTopOf="parent"
29         app:layout_constraintBottom_toBottomOf="parent"
30         android:contentDescription="@string/app_name" />
31
32     <ImageButton
33         android:id="@+id/btn_favorite"
34         android:layout_width="24dp"
35         android:layout_height="24dp"
36         android:layout_marginStart="4dp"
37         android:layout_marginTop="4dp"
38         android:background="@android:color/transparent"
39         android:src="@drawable/ic_love_border"
40         android:contentDescription="Favorite"
41     app:layout_constraintStart_toStartOf="@id/img_item_photo"
42     app:layout_constraintTop_toTopOf="@id/img_item_photo" />
43
44     <TextView
45         android:id="@+id/tv_item_name"
46         android:layout_width="0dp"
47         android:layout_height="wrap_content"
48         android:text="Tas Kulit Elegan"
49         android:textColor="@color/black"
50         android:textStyle="bold"
51         android:textSize="20sp"
52         app:layout_constraintTop_toTopOf="parent"
53     app:layout_constraintStart_toEndOf="@id/img_item_photo"
54     app:layout_constraintEnd_toEndOf="parent"
55     app:layout_constraintHorizontal_bias="0.0"
56     android:layout_marginStart="8dp"
57     tools:ignore="MissingConstraints" />
58
59     <TextView
60         android:id="@+id/tv_item_price"
61         android:layout_width="0dp"
62         android:layout_height="wrap_content"
63         android:text="Rp 350.000"
64         android:textSize="16sp"
65         android:textColor="@color/black"
66         app:layout_constraintTop_toBottomOf="@id/tv_item_name"
67     app:layout_constraintStart_toStartOf="@id/tv_item_name"
68     app:layout_constraintEnd_toEndOf="parent"
69     android:layout_marginTop="4dp" />
70
71     <Button
72         android:id="@+id/button_shopee"
73         android:layout_width="wrap_content"
74         android:layout_height="wrap_content"
75         android:layout_marginStart="8dp"
76         android:layout_marginTop="40dp"
77         android:text="@string/btn_shopee"
78     app:layout_constraintStart_toStartOf="@id/tv_item_name"
79     app:layout_constraintTop_toBottomOf="@id/tv_item_price" />

```

80	
81	<Button
82	android:id="@+id/button_detail"
83	android:layout_width="wrap_content"
84	android:layout_height="wrap_content"
85	android:layout_marginStart="20dp"
86	android:text="@string/btn_detail"
87	app:layout_constraintStart_toEndOf="@id/button_shopee"
88	app:layout_constraintTop_toTopOf="@id/button_shopee" />
89	</androidx.constraintlayout.widget.ConstraintLayout>
90	</androidx.cardview.widget.CardView>

Tabel 24. Source Code Jawaban Soal 1

25. app/src/main/res/layout/fragment_detail.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<LinearLayout
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	android:layout_width="match_parent"
5	android:layout_height="match_parent"
6	android:orientation="vertical"
7	android:background="#F6F6F6">
8	
9	<androidx.appcompat.widget.Toolbar
10	android:id="@+id/toolbar"
11	android:layout_width="match_parent"
12	android:layout_height="?attr/actionBarSize"
13	android:background="#D1B3FF"
14	android:text="ShopBag - Your Bag Your Style"
15	android:textColor="@color/black"
16	android:theme="@style/ToolbarStyle"/>
17	
18	<ScrollView
	xmlns:android="http://schemas.android.com/apk/res/android"
19	xmlns:app="http://schemas.android.com/apk/res-auto"
20	android:layout_width="match_parent"
21	android:layout_height="match_parent"
22	android:padding="16dp"
23	android:background="#F6F6F6">
24	
25	<androidx.constraintlayout.widget.ConstraintLayout
26	android:layout_width="match_parent"
27	android:layout_height="wrap_content"
28	android:background="#F6F6F6">
29	
30	<ImageView
31	android:id="@+id/img_item_photo"
32	android:layout_width="350dp"
33	android:layout_height="400dp"
34	android:scaleType="centerCrop"

35	app:layout_constraintTop_toTopOf="parent"
36	app:layout_constraintStart_toStartOf="parent"
37	app:layout_constraintEnd_toEndOf="parent"
38	android:contentDescription="@string/app_name"
	/>
39	
40	<TextView
41	android:id="@+id/tv_name"
42	android:layout_width="wrap_content"
43	android:layout_height="wrap_content"
44	android:textStyle="bold"
45	android:textColor="@color/black"
46	android:textSize="25sp"
47	android:layout_marginTop="16dp"
48	app:layout_constraintTop_toBottomOf="@id/img_item_photo"
49	app:layout_constraintStart_toStartOf="parent"
50	app:layout_constraintEnd_toEndOf="parent"/>
51	
52	<TextView
53	android:id="@+id/tv_harga"
54	android:layout_width="wrap_content"
55	android:layout_height="wrap_content"
56	android:textColor="@android:color/holo_red_dark"
57	android:textStyle="bold"
58	android:textSize="18sp"
59	android:layout_marginTop="8dp"
60	app:layout_constraintTop_toBottomOf="@id/tv_name"
61	app:layout_constraintStart_toStartOf="parent"
62	app:layout_constraintEnd_toEndOf="parent" />
63	
64	<TextView
65	android:id="@+id/tv_deskripsi"
66	android:layout_width="match_parent"
67	android:layout_height="wrap_content"
68	android:layout_marginTop="30dp"
69	android:textSize="16sp"
70	android:textColor="@color/black"
71	android:background="#F6F6F6"
72	android:textAlignment="center"
73	android:lineSpacingExtra="5dp"
74	android:padding="8dp"
75	app:layout_constraintTop_toBottomOf="@id/tv_harga"
76	app:layout_constraintStart_toStartOf="parent"
77	app:layout_constraintEnd_toEndOf="parent" />
78	</androidx.constraintlayout.widget.ConstraintLayout>
79	</ScrollView>
80	</LinearLayout>

Tabel 25. Source Code Jawaban Soal 1

26. app/src/main/res/layout/activity_main.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <androidx.constraintlayout.widget.ConstraintLayout
3 xmlns:android="http://schemas.android.com/apk/res/android"
4   xmlns:tools="http://schemas.android.com/tools"
5   xmlns:app="http://schemas.android.com/apk/res-auto"
6   android:layout_width="match_parent"
7   android:layout_height="match_parent"
8   tools:context=".MainActivity"
9   tools:ignore="ExtraText">
10
11   <androidx.fragment.app.FragmentContainerView
12       android:id="@+id/nav_host_fragme
13   android:name="androidx.navigation.fragment.NavHostFragment"
14       android:layout_width="match_parent"
15       android:layout_height="match_parent"
16       app:navGraph="@navigation/nav_graph"
17       app:defaultNavHost="true" />
18 </androidx.constraintlayout.widget.ConstraintLayout>
```

Tabel 26. Source Code Jawaban Soal 1

27. app/src/main/res/navigation/nav_graph.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <navigation
3 xmlns:android="http://schemas.android.com/apk/res/android"
4   xmlns:app="http://schemas.android.com/apk/res-auto"
5   xmlns:tools="http://schemas.android.com/tools"
6   app:startDestination="@id/homeFragment">
7
8   <fragment
9       android:id="@+id/homeFragment"
10      android:name="com.example.modul5.ui.home.HomeFragment"
11      android:label="fragment_home"
12      tools:layout="@layout/fragment_home">
13       <action
14           android:id="@+id/action_homeFragment_to_detailFragment"
15           app:destination="@id/detailFragment" />
16     </fragment>
17
18     <fragment
19         android:id="@+id/detailFragment"
20         android:name="com.example.modul5.ui.detail.DetailFragment"
21         android:label="fragment_detail"
22         tools:layout="@layout/fragment_detail" />
23 </navigation>
```

Tabel 27. Source Code Jawaban Soal 1

28. app/src/main/res/values/strings.xml

```
1 <resources>
2     <string name="app_name">MODUL5</string>
3     <string name="label_harga">Harga</string>
4     <string name="label_deskripsi">Deskripsi</string>
5     <string name="btn_detail">Detail</string>
6     <string name="btn_shopee">Shopee</string>
7 </resources>
```

Tabel 28. Source Code Jawaban Soal 1

29. app/src/main/res/values/styles.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <resources>
3     <style name="ToolbarStyle"
4         parent="ThemeOverlay.AppCompat.Light">
5         <item
6             name="android:textColorPrimary">@color/black</item>
7         <item name="android:textStyle">bold</item>
8         <item
9             name="titleTextAppearance">@style/ToolbarTitleText</item>
10        </style>
11
12        <style name="ToolbarTitleText"
13            parent="TextAppearance.Widget.AppCompat.Toolbar.Title">
14            <item name="android:textColor">@color/black</item>
15            <item name="android:textStyle">bold</item>
16            <item name="android:textSize">25sp</item>
17        </style>
18    </resources>
```

Tabel 29. Source Code Jawaban Soal 1

30. Data JSON dari Beeceptor

```
[
  {
    "id": 1,
    "nama": "BOSTANTEN [Amora Bag]",
    "harga": "Rp159.999",
    "gambar": "https://down-id.img.susercontent.com/file/id-11134207-7rbkb-m9jogtk5msugdb.webp",
    "shopeeLink": "https://id.shp.ee/vKZm8c9",
    "deskripsi": "(Bostanten) 100% asli. Selamat datang di toko Bostanten, di mana kami akan berusaha untuk menyediakan produk yang memenuhi kebutuhan kualitas dan efisiensi kepada pelanggan. Beli produk kami dengan mudah dari pusat perbelanjaan Shopee karena kami menjamin keaslian 100%. Tas ini terbuat dari bahan kulit PU dan tersedia dalam pilihan warna Off Putih, Hitam, Coklat, dan Aprikot. Dengan dimensi 22.5 cm (Lebar) × 26.5 cm
```

(Tinggi) × 3.5 cm (Tebal) dan berat 0.28 kg, tas ini dilengkapi dengan ruang penyimpanan besar, satu kantong, dan satu kantong resleting. Tali belakangnya memiliki panjang antara 103-124 cm. Tas ini cocok digunakan untuk sekolah, kantor, perjalanan, perjalanan bisnis, belanja, pertemuan, makan, kencan, serta acara sosial seperti pesta dan konser. Target penggunaanya ialah wanita, gadis, pekerja kantoran, pelajar, dan remaja."

```

    },
    {
      "id": 2,
      "nama": "BOSTANTEN [LYN Bag]",
      "harga": "Rp159.999",
      "gambar": "https://down-id.img.susercontent.com/file/id-11134207-7rbk7-m9jl9r3bsgl780.webp",
      "shopeeLink": "https://id.shp.ee/56fWCro",
      "deskripsi": "(Bostanten) 100% Asli. Selamat datang di toko Bostanten, di mana kami akan berusaha untuk menyediakan produk yang memenuhi kebutuhan kualitas dan efisiensi kepada pelanggan. Beli produk kami dengan mudah dari pusat perbelanjaan Shopee karena kami menjamin keaslian 100%. Tas ini terbuat dari bahan kulit PU dengan pilihan warna Coklat, Off Putih, Hitam, dan Aprikot Coklat. Memiliki dimensi 29 cm (Lebar) × 14 cm (Tinggi) × 9 cm (Tebal), dan berat sekitar 0.34 kg. Bagian dalam terdiri dari ruang penyimpanan besar, satu tas masukan dalam, dan satu tas zipper internal. Pegangan tas memiliki jarak sedang yaitu 31 cm. Sangat cocok digunakan untuk sekolah, kantor, pesta, dan perjalanan."
    },
    {
      "id": 3,
      "nama": "BOSTANTEN [Madeline Bag]",
      "harga": "Rp179.999",
      "gambar": "https://down-id.img.susercontent.com/file/id-11134207-7rbk0-m9jkl0819vql50.webp",
      "shopeeLink": "https://id.shp.ee/7ARV68N",
      "deskripsi": "<item>(Bostanten) 100% asli. Selamat datang di toko Bostanten, di mana kami akan berusaha untuk menyediakan produk yang memenuhi kebutuhan kualitas dan efisiensi kepada pelanggan. Beli produk kami dengan mudah dari pusat perbelanjaan Shopee karena kami menjamin keaslian 100%. Tas ini terbuat dari bahan kulit PU dan tersedia dalam warna Off Putih, Warna Kamel, Hitam, Abu-abu, dan Coklat. Berukuran 28.5 cm (Lebar) × 21.5 cm (Tinggi) × 7 cm (Tebal), dengan berat 0.29 kg. Bagian dalam terdiri dari ruang penyimpanan besar, satu tas zipper belakang, dan satu saku masukan depan. Sabuk bahunya memiliki tinggi tengah sekitar 0-111 cm dan tidak dapat disesuaikan maupun dilepas. Tas ini cocok digunakan untuk sekolah, kantor, pesta, maupun perjalanan."
    },
    {
      "id": 4,
      "nama": "BOSTANTEN [Reigny Bag]",

```

```

    "harga": "Rp189.999",
    "gambar": "https://down-id.img.susercontent.com/file/id-11134207-7rbk0-m9nvrvmvwted736@resize_w900_nl.webp",
    "shopeeLink": "https://id.shp.ee/957MHuZ",
    "deskripsi": "(Bostanten) 100% asli. Selamat datang di toko Bostanten, di mana kami akan berusaha untuk menyediakan produk yang memenuhi kebutuhan kualitas dan efisiensi kepada pelanggan. Beli produk kami dengan mudah dari pusat perbelanjaan Shopee karena kami menjamin keaslian 100%. Tas ini menggunakan bahan kulit PU dan tersedia dalam warna Off Putih, Warna Kamel, dan Hitam. Ukurannya adalah 20 cm (Lebar) x 16 cm (Tinggi) x 9 cm (Tebal), dengan berat 0.27 kg. Dilengkapi dengan ruang penyimpanan besar, tas masukan, dan sabuk bahu dengan tinggi tengah sekitar 92-112 cm yang dapat disesuaikan dan dilepas pasang. Pegangan tas berukuran sedang, yaitu 9 cm. Cocok untuk sekolah, kantor, pesta, maupun perjalanan."
  },
  {
    "id": 5,
    "nama": "MOSSDOOM [Delphine Bag]",
    "harga": "Rp153.900",
    "gambar": "https://down-id.img.susercontent.com/file/id-11134207-7rasl-m63hsk5ln0ao17@resize_w900_nl.webp",
    "shopeeLink": "https://id.shp.ee/GaVtJQ1",
    "deskripsi": "Selamat datang di official MOSSDOOM store, 100% menjamin keaslian brand produk kami. Tas ini terbuat dari bahan kulit PU. Dengan ukuran 17 cm (Panjang) x 9 cm (Lebar) x 10 cm (Tinggi) dan berat 0.29 kg. Tersedia dengan beberapa pilhan warna Pink, Blue, Purple, Apricot, Beige White, Black. Tas ini memiliki struktur internal dengan saku penyimpanan besar, tinggi tali bahunya sekitar 110-121 cm (yang dapat disesuaikan dan bisa dilepas pasang). Semua foto produk kami adalah foto asli, diambil dari store official. Jadi dijamin keasliannya."
  },
  {
    "id": 6,
    "nama": "MOSSDOOM [Gradien Hand Bag]",
    "harga": "Rp249.000",
    "gambar": "https://down-id.img.susercontent.com/file/id-11134207-7rbk3-m8a5q159vrpp01.webp",
    "shopeeLink": "https://id.shp.ee/6tzCzpz",
    "deskripsi": "Selamat datang di produk MOSSDOOM, dijamin keasliannya 100%! Tas ini terbuat dari bahan kulit PU. Dengan ukuran panjang 19.5 cm, lebar 8 cm, tinggi 16.5 cm, dan dengan berat 0.36 kg. Tersedia hanya warna Pink. Tas ini dilengkapi dengan satu saku penyimpanan besar, satu saku sisipan, satu saku beritsleting, dan tinggi pegangan tas tangan sebesar 7cm. Untuk tali bahu memiliki panjang antara 47.5-59cm (dapat disesuaikan). Semua gambar produk ini diambil dalam kehidupan nyata, jadi dijamin keaslinya."
  },
  {

```

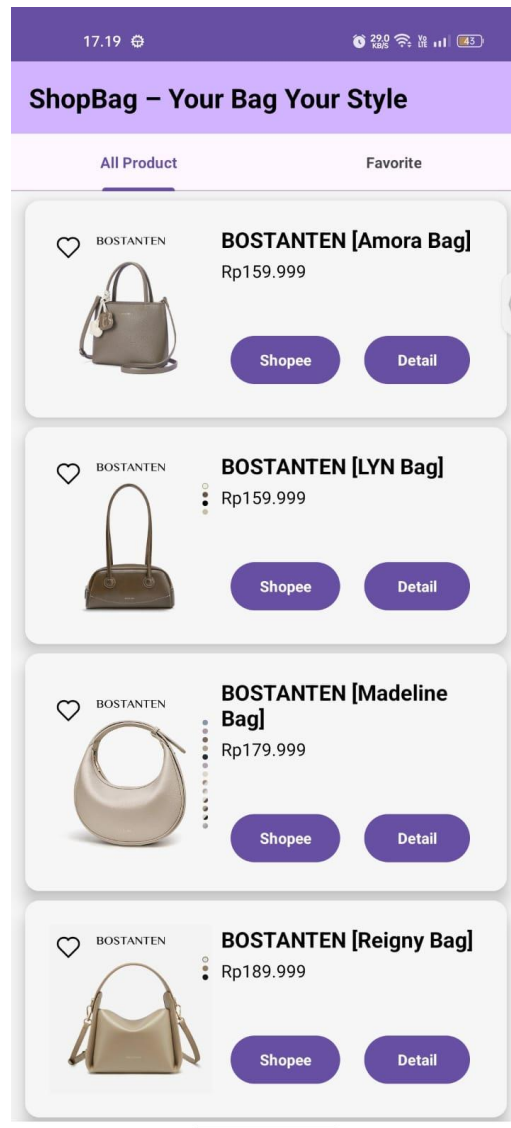
```

    "id": 7,
    "nama": "MOSSDOOM [Lysmare Bag]",
    "harga": "Rp189.000",
    "gambar": "https://down-id.img.susercontent.com/file/id-11134207-7rase-m63hsk5lptfkee.webp",
    "shopeeLink": "https://id.shp.ee/t3osSMq",
    "deskripsi": "Selamat datang di official MOSSDOOM store, 100%
    menjamin keaslian brand produk kami. Tas ini terbuat dari bahan
    kulit PU. Berukuran 20 cm (Panjang) x 8 cm (Lebar) x 14 cm
    (Tinggi) dan berat 0.36 kg. Tas ini tersedia dalam pilihan warna
    Blue, Light Pink, Pink, Brown, Beige White, Black. Tas ini
    dilengkapi dengan ruang penyimpanan besar, satu saku sisipan,
    satu saku beresleting, dan dengan tali bahu setinggi kira-kira
    106-130cm (yang dapat disesuaikan, bisa dilepas pasang). Semua
    foto produk kami adalah foto asli, diambil dari store official.
    Jadi dijamin keasliannya."
  }
]

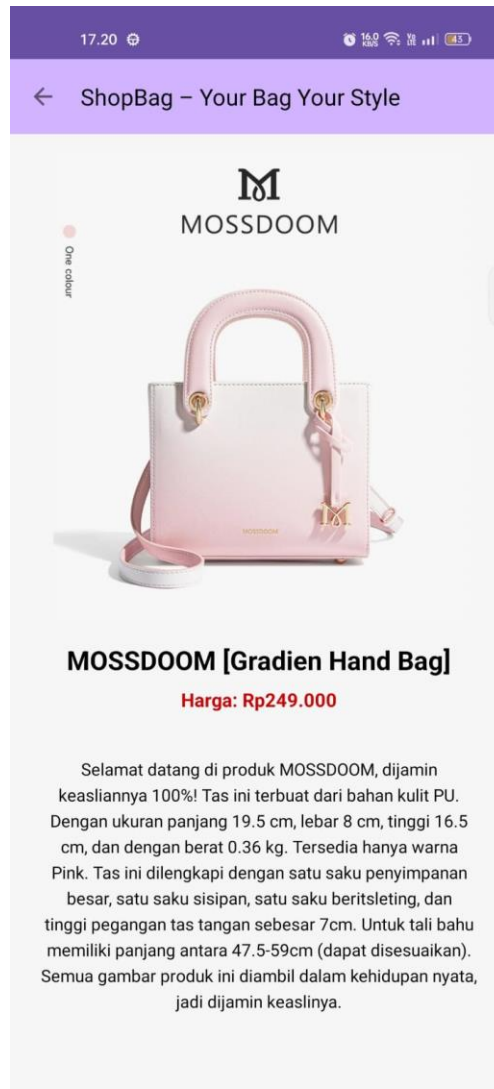
```

Tabel 30. Source Code Jawaban Soal 1

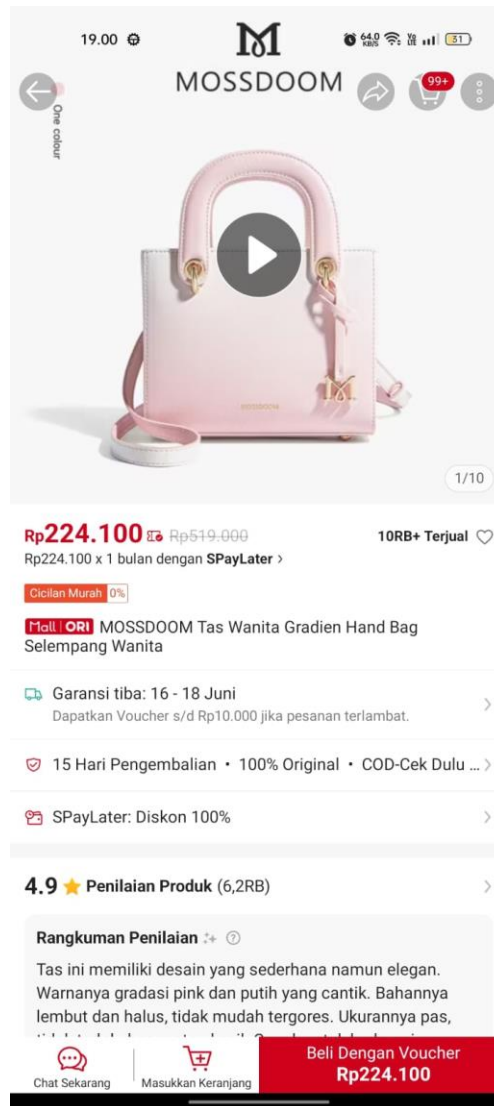
B. Output Program



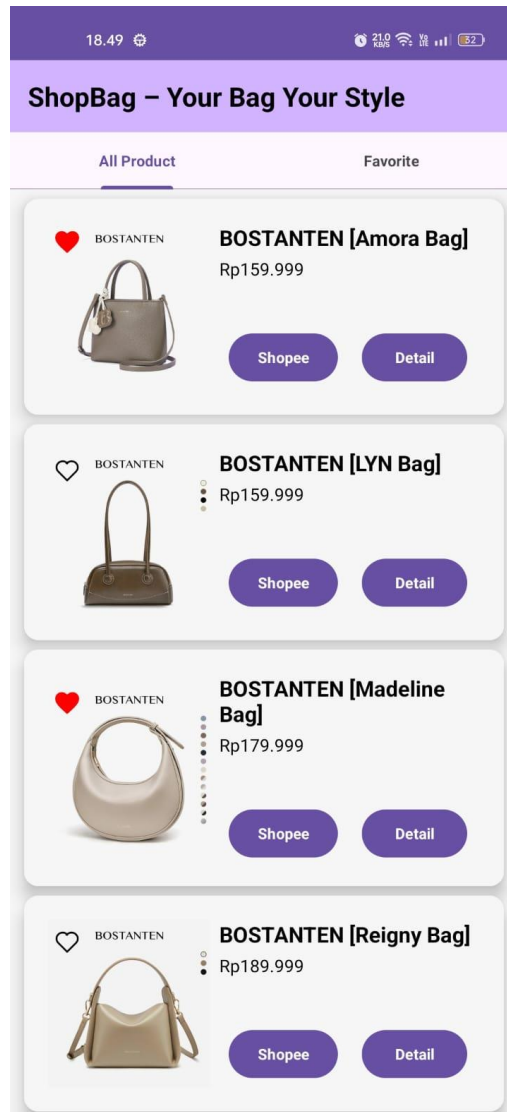
Gambar 1. Screenshot Hasil Jawaban Soal 1 UI List



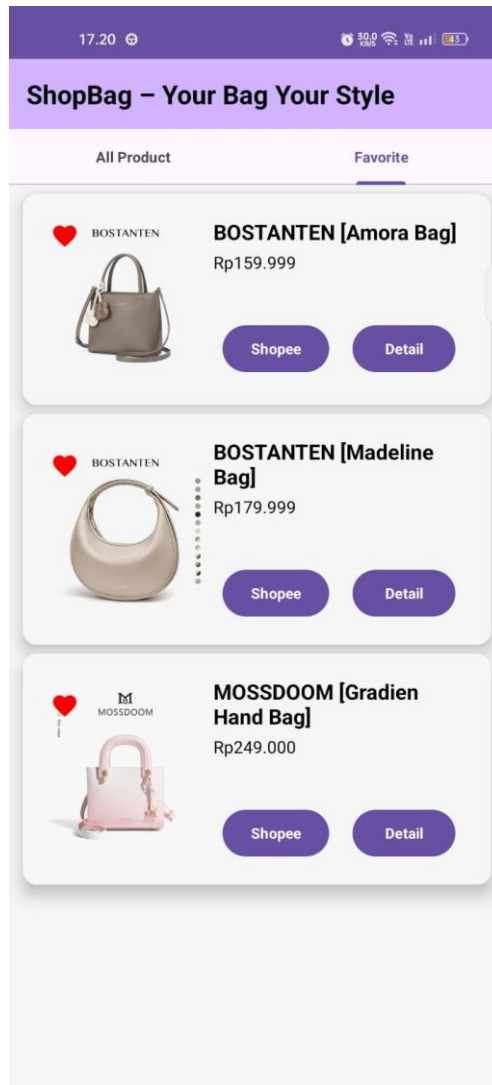
Gambar 2. Screenshot Hasil Jawaban Soal 1 UI Detail Saat Menekan Tombol Detail



Gambar 3. Screenshot Hasil Jawaban Soal 1 Berpindah ke Aplikasi Shopee Saat Menekan Tombol Shopee



Gambar 4. Screenshot Hasil Jawaban Soal 1 UI List Tambah Favorit



Gambar 5. Screenshot Hasil Jawaban Soal 1 UI Halaman Favorit

C. Pembahasan

1. ApiClient.kt

Pada line 1, `package com.example.modul5.data.api` digunakan untuk menandakan file ini berada di dalam package `data.api` dalam modul `modul5`. Kemudian, pada line 3 sampai 6, mengimpor konverter untuk `kotlinx.serialization`, konfigurasi format JSON, tipe `MediaType` dari `OkHttp` (digunakan saat menentukan `Content-Type`), dan retrofit builder class. Lalu, line 8 membuat singleton object `ApiClient` agar hanya ada satu instance Retrofit yang digunakan aplikasi. Line 9 terdapat URL dasar API yang akan digunakan Retrofit, disini saya menggunakan Beeceptor (endpoint <https://modul5.free.beeceptor.com/data>). Line 11 mengkonfigurasi parser JSON dari `kotlinx.serialization`. Opsi `ignoreUnknownKeys = true` berarti kalau ada field tambahan di JSON yang tidak dikenali, diabaikan saja (tidak menyebabkan error). Line 13 sampai 18 membuat instance Retrofit dengan menentukan `baseUrl` dari API, menentukan konverter JSON yang akan digunakan oleh Retrofit yaitu `kotlinx.serialization`, membangun instance Retrofit, dan menghubungkan Retrofit dengan interface `TasApiService`, hasil akhirnya adalah objek siap pakai untuk memanggil API `getTas()`.

2. TasApiService.kt

Pada line 1, `package com.example.modul5.data.api` digunakan untuk menandakan bahwa file ini berada di package `data.api`. Kemudian, pada line 3 dan 4, mengimpor anotasi Retrofit `@GET` dan model `Tas`. Lalu, line 6 mendefinisikan interface API. Ini akan digunakan oleh Retrofit untuk memanggil endpoint. Line 7 sampai 9 terdapat anotasi Retrofit yang artinya saat fungsi `getTas()` dipanggil, akan melakukan GET ke `BASE_URL + "data"` (<https://modul5.free.beeceptor.com/data>). Juga terdapat fungsi bersifat suspend (dipanggil dari coroutine). Ini akan memanggil endpoint API dan mengembalikan list of `tas` dalam bentuk `TasDto`.

3. TasDto.kt

Pada line 1, `package com.example.modul5.data.api` digunakan untuk menandakan file ini berada di package yang sama dengan service. Line 3 mengimpor anotasi `@Serializable` agar Kotlin bisa melakukan parsing JSON menggunakan `kotlinx.serialization`. Line 5, `@Serializable` menandai bahwa kelas `TasDto` bisa dikonversi dari dan ke JSON. Line 6 sampai 13 adalah struktur data tas dari API. Properti ini sesuai dengan isi dari JSON response yang disimpan di `Beeceptor`.

4. TasDao.kt

DAO (*Data Access Object*), yaitu interface untuk akses database. Line 10 dan 11 `@Dao interface TasDao {` berguna untuk memberi tahu Room bahwa ini adalah interface DAO yang akan digunakan untuk akses data `Tas`. Line 12 dan 13 mengambil semua data dari tabel `tas` sebagai `Flow`, artinya data bisa terus mengalir (*reactive*) dan ini cocok dipakai di `ViewModel`. Line 15 dan 16, memasukkan banyak data ke tabel `tas`. Jika `id` bentrok, akan menggantikan data lama (`REPLACE`). Line 18 sampai 20, memperbarui status `isFavorite` berdasarkan `id` `tas` tertentu.

5. TasDatabase.kt

Kelas `TasDatabase` ini ialah kelas abstrak yang menjadi basis Room database. Line 8 dan 9 menandakan ini adalah Room Database. `Entities` ialah daftar tabel yang digunakan (`TasEntity`), `version = 1` itu versi database, dan `exportSchema = false` artinya tidak mengeksport struktur database ke file `.json`. Line 10 terdapat fungsi abstrak untuk mendapatkan DAO (`TasDao`), inilah yang digunakan di `Repository` nantinya. Line 12 sampai 14, `INSTANCE` dibuat `@Volatile` supaya perubahan langsung terbaca oleh thread lain. Line 16 sampai 28, memastikan hanya satu instance database yang dibuat, sehingga aman dari multi-

threading. Juga membuat instance Room database bernama "tas_db". Lalu, Simpan instance dan kembalikan ke pemanggil.

6. TasEntity.kt

Kelas `TasEntity` ini ialah data model yang disimpan di database. Line 6 dan 7 menandakan ini adalah entitas/tabel bernama `tas` dalam database Room. Line 8, kolom `id` adalah primary key. Line 9 sampai 15 terdapat field untuk menyimpan informasi tas dan juga menyimpan status favorite untuk masing-masing tas.

7. TasMapper.kt

File `TasMapper` berisi tiga fungsi ekstensi penting, yaitu `TasDto.toEntity()` digunakan untuk mengubah objek dari API (`TasDto`) ke entitas database (`TasEntity`) dengan default `isFavorite = false`. `TasEntity.toDomain()` digunakan untuk mengubah data dari Room menjadi objek domain `Tas`, yang bisa digunakan oleh `ViewModel` dan ditampilkan di UI. Fungsi opsional (yang tidak ditulis di sini tapi bisa ditambahkan) seperti `Tas.toEntity()` kadang digunakan jika aplikasi ingin menyimpan perubahan dari UI ke database.

8. TasRepository.kt

Kelas ini berfungsi sebagai sumber data tunggal untuk `ViewModel`. Dengan menggabungkan data dari remote (API) dan local (Room). Fungsi `refreshFromApi()` hanya mengambil data dari server jika Room masih kosong, untuk menghindari hilangnya data lokal seperti favorit dan `toggleFavorite()` digunakan untuk mengubah status favorit suatu tas di database.

9. Tas.kt

Kelas ini adalah model utama yang digunakan di seluruh aplikasi, termasuk UI. Ditandai dengan `@Parcelize` agar bisa dikirim antar-fragment atau Activity. Penggunaan `@Transient` ialah untuk mencegah `isFavorite` ikut diserialisasi

saat parsing JSON dari API karena nilai ini hanya disimpan lokal (Room). Ini juga berfungsi sebagai bentuk final data sebelum ditampilkan ke pengguna.

10. AdapterTas.kt

AdapterTas adalah kelas `RecyclerView.Adapter` yang digunakan untuk menampilkan daftar produk tas di aplikasi. Adapter ini menerima tiga parameter berupa lambda function, masing-masing untuk menangani klik pada tombol detail (`onDetailClick`), tombol Shopee (`onShopeeClick`), dan tombol favorit (`onFavoriteClick`). Adapter menyimpan daftar data tas dalam properti `tasList`, yang awalnya kosong dan dapat diperbarui melalui fungsi `updateList`, lalu memanggil `notifyDataSetChanged()` agar daftar diperbarui di tampilan.

Di dalamnya terdapat `TasViewHolder`, inner class yang menyimpan binding terhadap layout `item_tas.xml` melalui `ViewBinding` (`ItemTasBinding`). Fungsi `onCreateViewHolder` bertugas meng-inflate layout item dan mengembalikan instance `TasViewHolder`. Saat setiap item di-bind ke tampilan melalui `onBindViewHolder`, data dari objek Tas ditampilkan ke `TextView` dan `ImageView` yang sesuai, termasuk memuat gambar dari URL menggunakan library `Coil`. Tombol love (favorit) akan menampilkan ikon berbeda tergantung status `isFavorite`, dan tiap tombol disambungkan ke callback fungsinya masing-masing.

Terakhir, `getItemCount` mengembalikan jumlah item yang akan ditampilkan oleh `RecyclerView`, yaitu panjang dari `tasList`. Keseluruhan kelas ini berfungsi sebagai penghubung antara data model tas dan tampilan daftar produk, serta menyediakan interaksi pengguna secara fleksibel dengan memanfaatkan callback dari luar..

11. HomePagerAdapter.kt

Kelas ini merupakan implementasi dari sebuah adapter bernama `HomePagerAdapter` yang digunakan untuk mengelola tampilan tab atau halaman pada aplikasi Android menggunakan `ViewPager2`. Adapter ini mewarisi kelas

FragmentManager, yang memungkinkan fragment ditampilkan dan dikelola secara efisien dalam ViewPager2. Konstruktor dari adapter ini menerima sebuah objek Fragment sebagai parameter konteks utama, biasanya fragment induk tempat ViewPager2 berada. Metode getItemCount() mengembalikan jumlah total halaman, yaitu 2, yang berarti terdapat dua tab yang akan ditampilkan. Metode createFragment(position: Int) bertanggung jawab untuk mengembalikan fragment yang sesuai berdasarkan posisi halaman. Jika posisi adalah 0, maka akan ditampilkan AllProductFragment, dan jika posisi adalah 1, maka akan ditampilkan FavoriteFragment. Apabila posisi yang diberikan tidak sesuai dengan rentang yang diharapkan (selain 0 atau 1), maka akan dilemparkan pengecualian IllegalArgumentException untuk menandakan bahwa posisi tersebut tidak valid.

12. HomeFragment.kt

Kelas ini adalah implementasi dari sebuah fragment bernama HomeFragment dalam aplikasi Android berbasis arsitektur MVVM (Model-View-ViewModel). Fragment ini bertanggung jawab sebagai tampilan utama yang menampilkan dua tab: "All Product" dan "Favorite", dengan menggunakan ViewPager2 dan TabLayout. Pada bagian line awal, fragment mengambil instance dari TasViewModel menggunakan delegation by activityViewModels yang dibantu oleh TasViewModelFactory untuk menyediakan context. Binding view diatur menggunakan FragmentHomeBinding, yang mempermudah akses ke komponen UI yang didefinisikan dalam layout XML terkait.

Dalam metode onCreateView, layout fragment di-inflate dan disimpan ke dalam _binding, lalu binding.root dikembalikan sebagai tampilan utama fragment. Kemudian, di onViewCreated, adapter HomePagerAdapter diinisialisasi dengan referensi this (fragment saat ini), dan dipasangkan ke ViewPager untuk mengatur halaman/tab. TabLayoutMediator digunakan untuk menghubungkan TabLayout dengan ViewPager, dan memberi label pada

masing-masing tab sesuai dengan posisi “All Product” di posisi 0 dan “Favorite” di posisi 1.

Selanjutnya, terdapat blok `lifecycleScope.launch` yang menjalankan coroutine untuk mengamati `navigateToDetail`, sebuah Flow yang dipancarkan oleh `viewModel`. Ketika `navigateToDetail` memancarkan nilai, fragment akan memeriksa apakah ada tas yang dipilih (`selectedTas`). Jika ada, maka navigasi akan dilakukan ke `DetailFragment` menggunakan `findNavController()`. Jika tidak ada tas yang dipilih, maka akan dicetak log error.

Terakhir, di metode `onDestroyView`, binding di-*null*-kan untuk menghindari memory leak ketika fragment dihancurkan. Jadi, fragment ini menangani tampilan dua kategori produk dan memungkinkan navigasi ke detail produk melalui `ViewModel` secara reaktif.

13. AllProductFragment.kt

Kelas ini adalah implementasi dari `AllProductFragment`, yaitu salah satu tab pada halaman utama aplikasi yang berfungsi untuk menampilkan daftar semua produk tas. Fragment ini menggunakan arsitektur MVVM dengan bantuan `ViewModel` (`TasViewModel`) dan `LiveData` berbasis Flow untuk memantau data produk secara reaktif. Binding view dilakukan menggunakan `FragmentAllProductBinding`, yang secara otomatis menghubungkan komponen UI dari layout XML ke kode Kotlin. `ViewModel` diperoleh menggunakan `delegation by activityViewModels` agar dapat berbagi data dengan fragment lainnya dalam satu aktivitas.

Pada `onCreateView`, layout di-inflate dan disimpan dalam `_binding`, lalu `binding.root` dikembalikan sebagai tampilan utama fragment. Di `onViewCreated`, adapter `AdapterTas` diinisialisasi dengan tiga parameter fungsi lambda, diantaranya yaitu `onDetailClick` digunakan untuk memanggil `selectTas(it)` dari `viewModel` ketika pengguna memilih produk untuk melihat detailnya, `onShopeeClick` digunakan untuk membuka link Shopee

menggunakan `Intent.ACTION_VIEW` ke browser atau aplikasi Shopee, berdasarkan `shopeeLink` dari produk, dan `onFavoriteClick` untuk memanggil `toggleFavorite(it)` untuk menandai atau menghapus tanda favorit pada produk tersebut. `RecyclerView` diatur menggunakan `LinearLayoutManager` agar menampilkan daftar secara vertikal, dan adapter dipasang ke `recyclerViewAll`.

Terakhir, di dalam `lifecycleScope.launch`, fragment mengamati `tasList`, yaitu aliran data dari `ViewModel`. Ketika data produk berubah, adapter akan diperbarui dengan `adapter.updateList(list)` untuk menampilkan daftar terbaru di layar. Saat fragment dihancurkan, binding di-set ke `null` di `onDestroyView` untuk mencegah memory leak. Secara keseluruhan, fragment ini menangani logika tampilan dan interaksi untuk semua produk, termasuk menampilkan daftar, membuka tautan eksternal, dan mengelola status favorit.

14. FavoriteFragment.kt

Kelas ini adalah implementasi dari `FavoriteFragment`, yaitu salah satu tab dalam tampilan utama aplikasi yang secara khusus menampilkan daftar produk tas yang telah ditandai sebagai favorit oleh pengguna. Fragment ini juga mengikuti arsitektur MVVM dan memanfaatkan `TasViewModel` sebagai sumber data yang dibagikan antar fragment. Fragment ini menggunakan view binding (`FragmentFavoriteBinding`) untuk mengakses elemen UI dari layout XML-nya. `ViewModel` diperoleh melalui delegation by `activityViewModels` dengan `TasViewModelFactory` sebagai penyedia context, sehingga `FavoriteFragment` berbagi `ViewModel` yang sama dengan fragment lainnya seperti `AllProductFragment`.

Di dalam metode `onCreateView`, layout di-inflate dan root view dikembalikan. Kemudian, pada `onViewCreated`, adapter `AdapterTas` diinisialisasi dengan tiga aksi, diantaranya yaitu `onDetailClick` digunakan saat item produk diklik, fragment akan memanggil `viewModel.selectTas(it)` untuk memilih produk tersebut agar nanti bisa diproses navigasi ke halaman detail,

`onShopeeClick` digunakan ketika pengguna menekan tombol beli, aplikasi membuka tautan Shopee dari produk tersebut menggunakan `Intent.ACTION_VIEW`, dan `onFavoriteClick` untuk menandai atau membatalkan status favorit pada item produk dengan memanggil `viewModel.toggleFavorite(it)`. `RecyclerView` pada layout (`recyclerViewFavorite`) dikonfigurasi dengan `LinearLayoutManager` agar menampilkan daftar produk secara vertikal, dan adapter yang telah dibuat sebelumnya dipasangkan ke dalamnya.

Selanjutnya, fragment menggunakan `coroutine` di dalam `viewLifecycleOwner.lifecycleScope.launch` untuk mengamati `tasList`, yaitu daftar produk dari `ViewModel`. Namun, berbeda dari `AllProductFragment`, di sini hanya produk yang memiliki atribut `isFavorite = true` yang akan ditampilkan. Proses ini dilakukan dengan menyaring daftar menggunakan `.filter { it.isFavorite }`, lalu memperbarui adapter dengan `adapter.updateList(favorites)`. Secara keseluruhan, fragment ini menangani logika tampilan untuk daftar produk favorit dan mendukung interaksi seperti membuka tautan Shopee, melihat detail produk, dan mengelola status favorit.

15. DetailFragment.kt

Kelas ini adalah implementasi dari `DetailFragment`, yaitu tampilan detail dalam aplikasi yang menampilkan informasi lengkap mengenai salah satu produk tas yang dipilih oleh pengguna. Fragment ini menggunakan arsitektur MVVM, dengan `TasViewModel` sebagai `ViewModel` yang mengelola data produk yang sedang dipilih (`selectedTas`). Pada awalnya, `ViewModel` diakses melalui `delegation by activityViewModels` menggunakan `TasViewModelFactory` agar `ViewModel` yang digunakan sama dengan fragment lain dalam aktivitas yang sama. `Binding` ke layout dilakukan menggunakan `FragmentDetailBinding`.

Di dalam metode `onCreateView`, layout di-inflate dan view root dikembalikan untuk ditampilkan. Kemudian, di `onViewCreated`, toolbar diatur

sebagai ActionBar menggunakan `setSupportActionBar`. Judul ActionBar diset menjadi "ShopBag – Your Bag Your Style", dan tombol navigasi balik (back) diaktifkan. Saat tombol tersebut ditekan, pengguna akan dinavigasikan kembali ke tampilan sebelumnya menggunakan `findNavController().navigateUp()`. Bagian inti dari fragment ini adalah coroutine dalam `lifecycleScope.launch` yang mengamati aliran data `selectedTas` dari ViewModel menggunakan `collectLatest`. Ketika data tas tersedia (tidak null), maka elemen UI seperti `TextView` dan `ImageView` akan diisi dengan informasi dari objek `tas`, yaitu nama, harga, deskripsi, dan gambar. Untuk memuat gambar, digunakan library Coil, yang menyediakan placeholder saat loading dan ikon error jika gambar gagal dimuat.

Jika `selectedTas` ternyata null (misalnya karena belum dipilih atau telah dihapus), pengguna secara otomatis dikembalikan ke halaman sebelumnya dengan `navigateUp()`, sebagai mekanisme perlindungan dari tampilan kosong atau kesalahan akses data. Terakhir, pada `onDestroyView`, method `clearSelectedTas()` dipanggil untuk menghapus data tas yang sedang dipilih dari ViewModel. Binding juga di-null-kan untuk menghindari memory leak. Sehingga, `DetailFragment` berfungsi untuk menampilkan detail produk secara dinamis berdasarkan pilihan pengguna, sekaligus menangani navigasi dan pengelolaan memori secara baik.

16. TasViewModel.kt

Kelas ini adalah implementasi dari `TasViewModel`, sebuah kelas yang berperan sebagai jembatan antara data (`TasRepository`) dan UI dalam arsitektur MVVM pada aplikasi Android. ViewModel ini digunakan untuk mengelola data tas, menangani aksi pengguna, dan menyimpan status sementara yang dibutuhkan antar fragment.

`val tasList: StateFlow<List<Tas>>` adalah aliran data utama yang berisi daftar produk tas. Nilai ini diperoleh dari `TasRepository`, lalu dikonversi menjadi `StateFlow` menggunakan `.stateIn()` agar data bisa diamati secara

berkelanjutan di UI. Fungsi `.onEach` digunakan di sini untuk mencetak jumlah data dari Room setiap kali data diperbarui, sebagai debug log. `_selectedTas` adalah `MutableStateFlow` yang menyimpan satu tas yang sedang dipilih oleh pengguna (misalnya untuk ditampilkan di `DetailFragment`). Properti publik `selectedTas` adalah versi hanya-baca dari nilai ini agar tidak bisa dimodifikasi langsung oleh kelas lain. `_navigateToDetail` adalah `MutableSharedFlow` yang digunakan untuk memicu event navigasi ke halaman detail. `SharedFlow` cocok untuk satu-kali event (seperti navigasi), karena tidak menyimpan nilai terakhir. `navigateToDetail` adalah versi publiknya.

Fungsi-fungsi penting seperti `init { refreshFromApi() }` ialah ketika `ViewModel` dibuat, ia langsung memanggil `refreshFromApi()` untuk mengambil data terbaru dari API dan menyimpannya ke database lokal. Fungsi `refreshFromApi()` ini memanggil repository untuk memuat ulang data dari API dalam coroutine. Jika terjadi error, pengecualian ditangani secara diam-diam. Fungsi `toggleFavorite(tas: Tas)` ini digunakan untuk menandai atau menghapus status favorit suatu produk tas berdasarkan id dan `isFavorite`-nya. `selectTas(item: Tas?)` berguna untuk menyimpan tas yang dipilih ke dalam `_selectedTas`, dan jika nilai tidak null, maka memicu navigasi ke halaman detail dengan memancarkan event `navigateToDetail`. `clearSelectedTas()` berguna untuk menghapus tas yang sedang dipilih dengan menyetel `_selectedTas` ke null. Biasanya dipanggil saat `DetailFragment` dihancurkan.

17. TasViewModelFactory.kt

Kelas ini adalah implementasi dari `TasViewModelFactory`, yaitu custom `ViewModelProvider.Factory` yang bertugas untuk membuat instance `TasViewModel` dengan parameter tambahan yang tidak bisa disediakan oleh konstruktor default, yaitu `Context`. `TasViewModelFactory` menerima `Context` dalam konstruktor. `ViewModelProvider.Factory` adalah antarmuka yang digunakan oleh `ViewModelProvider` untuk membuat

ViewModel secara manual. Fungsi `create` dipanggil saat framework (seperti `activityViewModels` atau `viewModels`) membutuhkan instance `ViewModel`, lalu memeriksa apakah `modelClass` adalah `TasViewModel`.

Line 12 sampai 18 adanya pembuatan logika `ViewModel`, yang dimana jika ya, maka mendapatkan instance database lokal (`TasDatabase`), lalu mengambil DAO-nya (`tasDao()`), mengakses API Retrofit melalui `ApiClient`, membuat objek `TasRepository` yang menjadi jembatan antara API dan database, dan mengembalikan instance `TasViewModel` dengan `repository` sebagai parameter. Jika kelas yang diminta bukan `TasViewModel`, maka `Exception` dilempar untuk mencegah pembuatan `ViewModel` yang tidak dikenal.

18. MainActivity.kt

`MainActivity` adalah aktivitas utama dalam aplikasi Android yang berfungsi sebagai titik masuk pertama saat aplikasi dijalankan. Kelas ini merupakan turunan dari `AppCompatActivity`, yang menyediakan dukungan kompatibilitas ke belakang untuk fitur-fitur Android modern. Dalam kelas ini digunakan `View Binding` untuk menghubungkan antara layout XML dan kode Kotlin secara lebih aman dan efisien tanpa perlu menggunakan `findViewById`. Properti binding dideklarasikan dengan `lateinit`, yang memungkinkan inisialisasi dilakukan nanti di dalam fungsi `onCreate()`. Di dalam `onCreate()`, layout utama aplikasi (`activity_main.xml`) di-inflate menggunakan `ActivityMainBinding.inflate(layoutInflater)`, kemudian ditampilkan melalui `setContentView(binding.root)`. Sehingga, seluruh komponen UI dalam layout tersebut bisa langsung diakses melalui objek `binding`, menjadikan kode aman dari `NullPointerException`, dan lebih mudah dibaca. Secara keseluruhan, `MainActivity` bertindak sebagai wadah awal bagi `fragment`-`fragment` yang ditampilkan melalui navigasi aplikasi.

19. ic_love_border.xml

XML ini ialah sebuah file vektor drawable yang digunakan untuk menggambarkan ikon berbentuk hati (love) dalam format gambar vektor pada Android. Vektor ini memiliki ukuran lebar dan tinggi masing-masing 24dp, serta menggunakan `viewportWidth` dan `viewportHeight` yang juga bernilai 24, yang artinya sistem koordinat internalnya dibuat sebanding dengan ukuran tampilannya. Di dalam elemen `<path>`, terdapat atribut `android:pathData` yang mendefinisikan bentuk dari ikon hati melalui serangkaian perintah vektor. Atribut `strokeColor="#000000"` untuk memberikan garis tepi berwarna hitam pada ikon, dan `strokeWidth="1.5"` digunakan untuk menetapkan ketebalan garis sebesar 1.5 satuan. Selain itu, `fillColor="@android:color/transparent"` membuat bagian dalam ikon transparan, sehingga hanya garis tepinya saja yang terlihat. Dengan konfigurasi ini, ikon hati akan tampil sebagai outline (garis luar) berbentuk hati tanpa isi warna di bagian dalamnya. Vektor ini digunakan untuk produk yang ingin dijadikan barang favoritnya.

20. ic_love_filled.xml

XML ini ialah sebuah file vector drawable pada Android yang menampilkan ikon hati (love) berwarna merah penuh. Vektor ini memiliki ukuran tampilan 24dp × 24dp dan sistem koordinat internal (`viewportWidth` dan `viewportHeight`) sebesar 24, yang umum digunakan untuk ikon standar. Di dalamnya terdapat elemen `<path>` yang mendefinisikan bentuk hati melalui atribut `android:pathData`, yaitu serangkaian perintah kurva dan garis untuk menggambar bentuk. Ikon ini memiliki warna isi merah solid (`android:fillColor="#FF0000"`) tanpa garis tepi, sehingga tampil sebagai gambar hati penuh. Ikon seperti ini umumnya digunakan untuk menyimbolkan item favorit atau disukai dalam antarmuka pengguna Android, dan berfungsi baik di berbagai ukuran layar karena menggunakan format vektor yang bersifat skalabel tanpa kehilangan kualitas.

21. fragment_home.xml

XML ini merupakan sebuah layout UI Android yang menggunakan `LinearLayout` sebagai kontainer utama dengan orientasi vertikal, artinya elemen-elemen di dalamnya akan ditampilkan dari atas ke bawah. Layout ini terdiri dari tiga komponen utama. Pertama, terdapat `TextView` yang berfungsi sebagai judul aplikasi dengan teks “ShopBag – Your Bag Your Style”. Komponen ini memiliki lebar penuh (`match_parent`), tinggi menyesuaikan isi (`wrap_content`), latar belakang berwarna ungu muda (`#D1B3FF`), padding sebesar 15dp agar teks tidak mepet ke pinggir, serta teks berwarna hitam, berukuran 25sp, dan bergaya tebal. Kedua, terdapat `TabLayout` dari pustaka Material Design yang akan digunakan untuk menampilkan tab navigasi, seperti “All Product” dan “Favorite”. Komponen ini biasanya digunakan bersama `ViewPager2` untuk membuat antarmuka tab yang interaktif.

Ketiga, ada `ViewPager2` yang merupakan komponen penampil halaman dengan scroll horizontal. `ViewPager` ini memiliki tinggi 0dp dan `layout_weight=1`, yang artinya ia akan mengisi sisa ruang yang tersedia di layar setelah `TextView` dan `TabLayout` ditampilkan. `ViewPager` ini akan menampung dan menampilkan fragment berdasarkan tab yang dipilih di `TabLayout`. Secara keseluruhan, layout ini dirancang untuk menampilkan halaman utama aplikasi dengan tampilan header, tab navigasi, dan konten yang dapat digeser antar tab.

22. fragment_all_product.xml

XML ini adalah layout Android yang menggunakan `LinearLayout` sebagai kontainer utama dengan orientasi vertikal dan latar belakang berwarna abu-abu terang (`#F6F6F6`). Di dalamnya hanya terdapat satu komponen utama, yaitu `RecyclerView`, yang diberi ID `recyclerViewAll`. `RecyclerView` ini berfungsi untuk menampilkan daftar item secara scrollable, seperti daftar produk pada toko online. Komponen ini mengisi seluruh lebar dan tinggi layar (`match_parent`), memiliki padding sebesar 4dp di sekelilingnya, dan atribut `clipToPadding="false"` memungkinkan isi `RecyclerView` untuk tetap terlihat saat menyentuh batas padding, misalnya untuk efek animasi atau shadow.

23. fragment_favorite.xml

XML ini adalah layout untuk tampilan Android yang menggunakan `LinearLayout` dengan orientasi vertikal dan latar belakang berwarna abu-abu terang (`#F6F6F6`). Di dalamnya terdapat sebuah `RecyclerView` dengan ID `recyclerViewFavorite`, yang digunakan untuk menampilkan daftar item favorit secara scrollable. `RecyclerView` ini mengisi seluruh ruang layar baik secara lebar maupun tinggi (`match_parent`), memiliki padding sebesar 4dp di sekelilingnya untuk memberikan jarak antar item dengan tepi layar, dan `clipToPadding="false"` agar item tetap bisa terlihat sampai ke area padding, misalnya untuk efek visual seperti bayangan atau animasi scroll. Layout ini digunakan sebagai bagian dari tab "Favorite" pada aplikasi yang menampilkan daftar produk favorit pengguna.

24. item_tas.xml

XML ini adalah layout item untuk sebuah daftar produk yang ditampilkan di dalam `RecyclerView`, menggunakan `CardView` sebagai kontainer utamanya. `CardView` memberikan efek bayangan (`elevation`) dan sudut melengkung (`cardCornerRadius`) agar tampilan produk terlihat seperti kartu yang elegan. Di dalam `CardView`, terdapat `ConstraintLayout` sebagai root view untuk mengatur posisi elemen secara fleksibel. Layout ini mencakup elemen-elemen `ImageView` (`img_item_photo`) untuk menampilkan gambar produk dengan ukuran tetap dan posisi di sisi kiri kartu. Gambar dipangkas (`centerCrop`) agar pas di dalam kotak. `ImageButton` (`btn_favorite`) digunakan untuk tombol berbentuk hati di pojok kiri atas gambar, yang berfungsi untuk menandai produk sebagai favorit. `TextView` (`tv_item_name`) untuk menampilkan nama produk di sebelah kanan gambar, menggunakan teks besar dan tebal. `TextView` (`tv_item_price`) untuk menampilkan harga produk di bawah nama produk. `Button` (`button_shopee`) ialah tombol yang mengarahkan pengguna ke tautan Shopee untuk membeli produk. `Button` (`button_detail`) ialah tombol untuk melihat detail lengkap produk.

25. fragment_detail.xml

XML ini merupakan tampilan antarmuka untuk halaman detail produk dalam aplikasi Android. Layout ini menggunakan struktur `LinearLayout` vertikal sebagai root, dengan latar belakang berwarna abu-abu muda (`#F6F6F6`). Di bagian atas, terdapat `Toolbar` yang berfungsi sebagai action bar dan menampilkan judul aplikasi “ShopBag – Your Bag Your Style” dengan latar ungu muda (`#D1B3FF`). Bagian isi utama dibungkus dalam `ScrollView`, yang memungkinkan konten digulir jika melebihi tinggi layar. Di dalam `ScrollView`, digunakan `ConstraintLayout` untuk mengatur posisi komponen secara presisi. Komponen utama meliputi `ImageView` (`img_item_photo`) digunakan untuk menampilkan gambar produk dengan ukuran besar (350x400dp) dan mode `centerCrop` agar gambar terisi penuh dan proporsional. `TextView` (`tv_name`) untuk menampilkan nama produk di bawah gambar, dengan huruf tebal dan ukuran besar (25sp). `TextView` (`tv_harga`) untuk menampilkan harga produk dengan warna merah gelap dan ukuran menonjol (18sp). `TextView` (`tv_deskripsi`) digunakan untuk menampilkan deskripsi produk dengan teks rata tengah, spasi baris ekstra, dan padding agar mudah dibaca.

26. nav_graph.xml

Pada line 1 sampai 5, menentukan bahwa fragment pertama kali yang ditampilkan saat app berjalan adalah `HomeFragment`. Line 7 sampai 11, mewakili untuk tampilan utama (daftar tas). Fragment ini berada di package `com.example.modul3`. Layout yang digunakan ialah `fragment_home.xml`. Line 12 sampai 15, mendefinisikan rute navigasi dari `HomeFragment` ke `DetailFragment`, hal ini akan dipicu saat user klik tombol “Detail” pada item tas. Line 17 sampai 22 adalah fragment untuk menampilkan detail dari item tas yang dipilih, dengan menggunakan layout `fragment_detail.xml`.

27. strings.xml

File XML tersebut adalah berkas `strings.xml` yang berfungsi untuk menyimpan resource string dalam aplikasi Android. File ini memungkinkan pengelolaan teks secara terpusat dan memudahkan penerjemahan (lokalisasi) ke berbagai bahasa. `<string name="app_name">MODUL5</string>` ialah endefinisikan nama aplikasi yang akan ditampilkan di launcher dan toolbar. `<string name="label_harga">Harga</string>` adalah label teks untuk menampilkan harga produk. `<string name="label_deskripsi">Deskripsi</string>` adalah label teks untuk bagian deskripsi produk. `<string name="btn_detail">Detail</string>` ialah teks pada tombol yang mengarahkan ke halaman detail produk. `<string name="btn_shopee">Shopee</string>` ialah teks pada tombol yang mengarahkan ke halaman Shopee.

28. styles.xml

File XML tersebut adalah file `styles.xml`, yang digunakan untuk mendefinisikan gaya (style) kustom dalam aplikasi Android. Style ini mempermudah penerapan tampilan yang konsisten di berbagai komponen UI. `<style name="ToolbarStyle" parent="ThemeOverlay.AppCompat.Light">` Style ini digunakan untuk mendesain tampilan Toolbar. Style ini mewarisi dari tema bawaan `AppCompat.Light`, yang merupakan gaya terang standar dari Android. `<item name="android:textColorPrimary">@color/black</item>` digunakan untuk mengatur warna utama teks dalam toolbar menjadi hitam. `<item name="android:textStyle">bold</item>` untuk menjadikan teks dalam toolbar tampil tebal (bold). `<item name="titleTextAppearance">@style/ToolbarTitleText</item>` ialah menentukan bahwa gaya teks judul toolbar akan menggunakan style `ToolbarTitleText`. `<style name="ToolbarTitleText">` Style ini digunakan secara spesifik untuk tampilan teks judul toolbar. `<item`

`name="android:textColor">@color/black</item>` untuk warna teks judul diset ke hitam. `<item name="android:textStyle">bold</item>` untuk teks judul ditampilkan dalam gaya tebal. `<item name="android:textSize">25sp</item>` ialah ukuran teks judul dengan ditetapkan sebesar 25sp (scale-independent pixels), memastikan keterbacaan pada berbagai ukuran layar.

29. activity_main.xml

Pada line 1 sampai 9, `ConstraintLayout` adalah layout utama yang digunakan di sini untuk mengatur posisi elemen UI di dalamnya. `ConstraintLayout` memungkinkan penataan elemen UI dengan fleksibilitas tinggi melalui constraint (penghubung). `xmlns:android` adalah deklarasi namespace untuk atribut Android standar. `xmlns:tools` digunakan oleh Android Studio untuk memberikan data atau tips visual saat pengembangan tanpa mempengaruhi aplikasi yang sebenarnya. `tools:context=".MainActivity"` memberikan konteks di mana layout ini digunakan (dalam hal ini, `MainActivity`). `tools:ignore="ExtraText"` memberikan petunjuk kepada Android Studio untuk mengabaikan peringatan tertentu yang tidak penting (misalnya teks ekstra).

Kemudian, line 11 sampai 18 terdapat `FragmentContainerView`, yang di mana komponen ini digunakan untuk menampilkan dan mengelola fragment di dalam activity. `FragmentContainerView` menggantikan penggunaan `FrameLayout` untuk penempatan fragment dan lebih terintegrasi dengan `Navigation Component` Android. `android:id="@+id/nav_host_fragme"` memberikan ID untuk `FragmentContainerView`, yang nantinya bisa digunakan untuk referensi dalam kode.

`android:name="androidx.navigation.fragment.NavHostFragment"` untuk menyatakan bahwa fragment yang akan ditampilkan di dalam `FragmentContainerView` adalah `NavHostFragment`, yang merupakan bagian dari `Navigation Component`. Fragment ini bertugas mengelola navigasi antar

fragment. `android:layout_width="match_parent"` dan `android:layout_height="match_parent"` digunakan untuk membuat `FragmentManager` mengambil ruang penuh dari layar, dengan lebar dan tinggi `match_parent`, sehingga mengisi seluruh layout. `app:navGraph="@navigation/nav_graph"` untuk menyediakan `NavGraph` yang berisi alur navigasi antar fragment. `@navigation/nav_graph` adalah referensi ke file `nav_graph.xml`, yang mendefinisikan destinasi dan aksi antar fragment dalam aplikasi. `app:defaultNavHost="true"` digunakan untuk menandakan bahwa `NavHostFragment` ini adalah host utama untuk navigasi di activity ini. Artinya, fragment ini akan menangani back stack navigasi, dan tombol back perangkat akan berfungsi untuk kembali ke fragment sebelumnya (jika ada).

D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

<https://github.com/nrhikmahhh99/Pemrograman-Mobile>